UGB Campus

API reference · v1

Every endpoint available today (94), with its goal, who may call it, the request it expects, what it returns and the errors it can raise.

Base URL	`http://localhost:3000/api/v1` (development)
Live docs	`http://localhost:3000/docs`: Swagger UI, also the temporary back office
Status	Built and tested: authentication, accounts, organizations, payments (Wave / Orange Money simulated), notifications and push, Sunu Taxi, chat, tickets and caravans (booking, transfer, boarding, offline scanning). Still to build: see the appendix.
Source	Generated from the OpenAPI spec of the running backend; every implemented route is covered.
Date	31 October 2026

Contents

How to use this API 5

Typical flows 6

Authentication 8

- Check that a phone and a student number are free . . . 8
- Send a verification code . . . 9
- Send a new code for the same verification . . . 9
- Check the code typed by the user . . . 10
- Create the student account . . . 11
- Sign in . . . 12
- Get a new access token . . . 12
- Sign out on this device . . . 13
- Sign out everywhere . . . 13
- Choose a new password (forgot password) . . . 14
- Change my password . . . 14

My account 16

- Who am I, and which spaces can I open? . . . 16
- Update where I live . . . 17
- Read my notification settings . . . 17
- Change my notification settings . . . 18
- Add or change my e-mail address . . . 18
- Remove my e-mail address . . . 19
- Save my current position . . . 19
- Forget my position . . . 19

Payments 21

- Read a payment . . . 21
- Money sent to me . . . 21
- Payouts to follow up (finance) . . . 22
- Payment page (simulator, development only) . . . 22
- Simulate the provider's answer (development only) . . . 23

Notifications 24

- Register this phone for push . . . 24
- Stop push on this phone . . . 24
- The bell list . . . 25
- The bell's dot . . . 25
- Mark as read . . . 25

Notify one account (back office) . . . 26

Sunu Taxi **27**

Open trips (the board) . . . 27
One trip . . . 28
My Sunu Taxi standing . . . 29
Propose a trip . . . 29
Take a seat and pay . . . 30
What leaving would refund . . . 30
Leave a trip . . . 31
Cancel my trip (organizer) . . . 31
Hand the trip over ("Passer le relais") . . . 32
"Je suis là" . . . 32
Say which taxi it is . . . 32
"On est partis" . . . 33
Collect the paid seats . . . 33

Chat **35**

My conversations and their badges . . . 35
One conversation . . . 35
Messages . . . 36
Send a message . . . 36
Mark as read . . . 37
Report a message . . . 37
Block the author . . . 38
Reported messages . . . 38

Caravans **39**

Departures to book . . . 39
One departure . . . 39
Seat map . . . 40
Book a seat and pay . . . 40
A booking (after paying) . . . 41
Release a seat I'm paying for . . . 42

Tickets **43**

My tickets . . . 43
One ticket, with its QR code . . . 43
Cancel my ticket . . . 44
Offer my ticket to a friend . . . 44
Offers waiting for an answer . . . 45

Accept a ticket . . . 45
Decline a ticket . . . 46
Withdraw my offer . . . 46
Organizer space **47**
My organization and my role in it . . . 47
Update my organization's contact e-mail . . . 48
List my team . . . 48
Add a team member or change their roles . . . 49
Remove a team member . . . 49
Organizer: departures and boarding **51**
"Mes activités" . . . 51
One of my departures . . . 52
Passenger list . . . 52
Take the list offline . . . 52
Scan a ticket ("Faire monter") . . . 53
Send scans made offline . . . 54
"Faire descendre" . . . 54
"Le bus repart de cet arrêt" . . . 55
Collect the money ("Récupérer") . . . 55
My vehicles . . . 56
Register a vehicle . . . 56
Create a departure (draft) . . . 57
Edit a draft . . . 58
Publish (open bookings) . . . 58
Cancel a departure . . . 59
Admin back office **60**
Create an account for an organizer or staff member . . . 60
Set a user's platform roles . . . 61
List organizations . . . 61
Create an organization . . . 62
Update an organization . . . 63
Give someone a role in an organization . . . 63
Remove someone from an organization . . . 64
Health **65**
Is the process alive? . . . 65
Can this instance serve traffic? . . . 65
Appendix: planned endpoints **66**

How to use this API

Format. Requests and responses are JSON (`Content-Type: application/json`). Unknown fields in a request body are dropped; fields of the wrong type are refused. Phone numbers are always Senegalese mobiles; the server normalizes them ("77 123 45 67" → +221771234567) and only ever returns them masked ("77 ••• •• 67").

Authentication. Sign-in returns an **access token** (valid 15 minutes) and a **refresh token** (valid 30 days of inactivity, 90 days at most). Send the access token on every protected call:

```
Authorization: Bearer eyJhbGciOiJFZERTQSIsImtpZCI6ImRldi0xIn0...
```

When a call answers **401**, call `POST /auth/refresh` once, store the *new* refresh token, and retry. If refresh also answers 401, send the user back to the login screen. Store the refresh token in secure storage (expo-secure-store), never in plain AsyncStorage.

Who may call what. Rights are always checked by the server, whatever the app shows. The access rules used below:

Access	Meaning
Public	No token needed.
Bearer token	Any signed-in user.
+ student profile	The account was created through student sign-up (organizer-only accounts get 403).
+ role in the organization	Active member of the organization in the URL with the listed role. Otherwise 404.
+ platform role admin	Back-office user. Checked against the database on each call, so a removed role stops working immediately.

Errors. Every error has the same shape (RFC 9457, `application/problem+json`). Use `code` in the app logic; `message` is French text that can be shown to the user as is. Some errors add fields (attemptsLeft, field, fields, retryAfter).

```
{
  "type": "about:blank#otp-invalid",
  "title": "OTP_INVALID",
  "status": 400,
  "code": "OTP_INVALID",
  "message": "Code incorrect.",
  "attemptsLeft": 3,
  "requestId": "39d69a58-3294-4aa6-90e3-d9ae75a7e170"
}
```

Errors that any endpoint can return, not repeated on each page:

Status	code	When
400	VALIDATION_ERROR	Body or parameters don't match the schema. "fields" lists the paths, e.g. [{ "path": "/identifier", "rule": "type" }].
401	UNAUTHENTICATED	Protected route without a valid access token (missing, expired, revoked).
403	FORBIDDEN	Signed in but not allowed (not a student, not admin…).
404	NOT_FOUND	Unknown route or resource, or a resource the caller may not see.
429	TOO_MANY_REQUESTS	Rate limit reached. Header Retry-After gives the seconds to wait.
500	INTERNAL_ERROR	Unexpected failure. Nothing internal is exposed; quote the requestId to the support.

Request id. Every response carries an `x-request-id` header (also in error bodies). The app may send its own `X-Request-Id` (8–64 letters, digits, hyphens) to follow a request through the logs.

Real time. One WebSocket per app: `ws://…/api/v1/ws`. Send `{"type":"auth","accessToken":"…"}` within 5 s (the server closes with code 4401 when the token is missing or expires: refresh it and reconnect), then subscribe with `{"type":"sub","topic":"conversation:<id>"}` or `event:<departureId>` (organizers). Each socket also receives its own `user:<id>` events (badges, tickets, notifications). Sending always goes through REST; after a reconnection, fetch what was missed.

Retries are safe. Purchases, chat messages and scans are idempotent: a request retried after a network error never pays, posts or boards twice.

Rate limits are counted per IP address, and for sensitive actions also per phone number or login identifier. They are shared by all server instances. The limits are given on each endpoint.

Development values. OTP code: **1234**. Demo accounts (password **Campus2026**): 77 000 00 02 (Awa Diop, student and organizer), 77 000 00 01 (Modou Khoulé, organizer), 77 000 00 03 (Cheikh Fall, scanner), 77 123 45 67 (Moussa Sow, student), 77 000 00 99 (admin).

Typical flows

Sign-up (the app's 3 steps):

```
1. POST /auth/register/check   { phone, studentNumber }        -> { available }
2. POST /auth/otp              { phone, purpose: "register" }  -> { challengeId }
   POST /auth/otp/{id}/verify  { code: "1234" }                -> { verificationToken }
   (POST /auth/otp/{id}/resend after 60 s if the SMS didn't arrive)
3. POST /auth/register         { verificationToken, profile }  -> { userId, tokens }
   GET  /me                                                    -> spaces: ["student"]
```

Sign-in and staying signed in:

```
POST /auth/login    { identifier, password }  -> { tokens }
GET  /me                                      -> which space to open (student / organizer)
...  any call with Authorization: Bearer <accessToken>
     401 ?  POST /auth/refresh { refreshToken } -> new tokens
            (store the new refresh token), retry once
```

Forgot password:

```
POST /auth/otp            { phone, purpose: "reset_password" }
POST /auth/otp/{id}/verify { code }  -> { verificationToken }
POST /auth/password/reset  { verificationToken, newPassword }
```

Buying a caravan ticket (same pattern for a Sunu Taxi seat):

```
GET  /caravans/departures?direction=aller     -> pick one
GET  /caravans/departures/{id}/seats          -> seat map, suggested seat
POST /caravans/departures/{id}/bookings
     { seat?, pickupStopId, dropoffStopId, method, returnUrl }
     -> { booking (held 10 min), payment: { reference, checkoutUrl } }
open checkoutUrl (Wave / Orange Money; simulator in development)
GET  /payments/{reference}       until it is no longer pending
GET  /caravans/bookings/{id}     -> { status: "confirmed", ticketId }
GET  /tickets/{ticketId}         -> signed QR code
```

Boarding (organizer, the day of the trip):

```
GET  /org/{orgId}/departures                    -> "Mes activités"
GET  /org/{orgId}/departures/{id}/offline-pack  -> list + QR key on the phone
POST /org/{orgId}/admissions { qr | code | ticketId, eventId, clientScanId }
     -> admitted / duplicate / rejected
     (no network: checked on the phone, sent later to /admissions/batch)
POST /org/{orgId}/departures/{id}/stops/p1/leave -> absent passengers
POST /org/{orgId}/departures/{id}/claim { method } -> the money (organization)
```

Onboarding an organizer (admin, from Swagger):

```
POST /admin/users                       { phone, firstName, lastName }
                                        (skip if they already have an account)
POST /admin/organizations               { slug, name, category, capabilities, email? }
PUT  /admin/organizations/{id}/members  { phone, roles: ["owner"] }
The organizer sets a password with "mot de passe oublié", signs in,
    and GET /me now lists "organizer".
```

Authentication

Sign-up (the app's 3 steps), login, sessions and passwords. Every route here is under `/api/v1/auth`. OTP codes are 4 digits; in development the code is always **1234** and is printed by the worker instead of being sent by SMS.

Method	Path	Goal
POST	/api/v1/auth/register/check	Check that a phone and a student number are free
POST	/api/v1/auth/otp	Send a verification code
POST	/api/v1/auth/otp/{challengeId}/resend	Send a new code for the same verification
POST	/api/v1/auth/otp/{challengeId}/verify	Check the code typed by the user
POST	/api/v1/auth/register	Create the student account
POST	/api/v1/auth/login	Sign in
POST	/api/v1/auth/refresh	Get a new access token
POST	/api/v1/auth/logout	Sign out on this device
POST	/api/v1/auth/logout-all	Sign out everywhere
POST	/api/v1/auth/password/reset	Choose a new password (forgot password)
PUT	/api/v1/auth/password	Change my password

Check that a phone and a student number are free

POST `/api/v1/auth/register/check`

Step 1 of sign-up ("Identification"). Tells the app whether the phone number or the student number already belongs to an account, so it can show the "Ce compte existe déjà" banner *before* sending a code. Nothing is created.

Access Public (no token)

Rate limit 10 per hour per IP

Request body (JSON)

Field	Type	Required	Rules / meaning
phone	string	yes	Senegalese mobile, 9 digits starting with 70, 71, 75, 76, 77 or 78. Spaces and the +221 / 00221 prefix are accepted: "77 123 45 67", "+221771234567".
studentNumber	string	yes	Format P + 6 digits, e.g. P232055. Case and spaces are ignored ("p 232055" works).

Example request

```
{ "phone": "77 123 45 67", "studentNumber": "P232055" }
```

Success response · 200 OK

```
{ "available": true }

// or, when taken:
{ "available": false, "conflict": "phone" }          // or "student_number"
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	Phone or student number badly formatted (field in the body).

Note. This route necessarily reveals whether a number is registered, which is why it is rate-limited.

Send a verification code

POST `/api/v1/auth/otp`

Starts a verification: generates a code for the phone and sends it by SMS. Used for sign-up step 2 (**register**) and for "mot de passe oublié" (**reset_password**). Returns a `challengeId` that the next calls refer to.

Access	Public (no token)
Rate limit	20 per hour per IP · 5 per day per phone number

Request body (JSON)

Field	Type	Required	Rules / meaning
`phone`	string	yes	Senegalese mobile, 9 digits starting with 70, 71, 75, 76, 77 or 78. Spaces and the +221 / 00221 prefix are accepted: "77 123 45 67", "+221771234567".
`purpose`	"register" | "reset_password"	yes	Why the code is requested.
`channel`	"sms" | "whatsapp" | "email"	no	Default "sms". Only SMS is available for now; the others return 422.

Example request

```
{ "phone": "775550101", "purpose": "register" }
```

Success response · 202 Accepted

```
{
  "challengeId": "6ae5dc3870d01ba6d4ae2f14",
  "expiresIn": 300,        // seconds the code stays valid
  "resendAfter": 60,       // seconds before a new code can be asked
  "codeLength": 4
}
```

Specific errors

Status	code	When
409	`PHONE_TAKEN`	purpose = register and the phone already has an account.
404	`ACCOUNT_NOT_FOUND`	purpose = reset_password and the phone has no account ("Aucun compte sur ce numéro."). No code is sent.
422	`CHANNEL_UNAVAILABLE`	WhatsApp or e-mail requested.
429	`TOO_MANY_REQUESTS`	Too many codes for this phone today, or from this IP.

Note. For **reset_password** the phone must have an account; for **register** it must not.

Send a new code for the same verification

POST `/api/v1/auth/otp/{challengeId}/resend`

"Renvoyer le code": replaces the code of an existing challenge (the previous code stops working) and sends it again. Also resets the attempt counter and the 5-minute expiry.

Access	Public (no token)
Rate limit	20 per hour per IP · 60 s between sends · 3 sends per challenge

Path parameters

Name	Type	Required	Description
`challengeId`	string (24 hex)	yes	Returned by POST /auth/otp.

Request body (JSON)

Field	Type	Required	Rules / meaning
`channel`	"sms" | "whatsapp" | "email"	no	Default "sms". Only SMS is available.

Example request

```
{}
```

Success response · 202 Accepted

```
Same body as POST /auth/otp.
```

Specific errors

Status	code	When
410	OTP_EXPIRED	Challenge unknown, expired, already used, or locked.
422	CHANNEL_UNAVAILABLE	WhatsApp or e-mail requested.
429	TOO_MANY_REQUESTS	Less than 60 s since the last send (header Retry-After), or 3 sends reached.

Check the code typed by the user

POST `/api/v1/auth/otp/{challengeId}/verify`

Compares the typed code with the challenge. On success, returns a **verificationToken**: a proof that this phone was verified, valid 15 minutes and usable **once**, which is then passed to POST /auth/register or POST /auth/password/reset. The answers mirror the app's VerifyResult (invalid with attemptsLeft / expired / locked).

Access Public (no token)

Rate limit 30 per 10 minutes per IP · 5 attempts per challenge

Path parameters

Name	Type	Required	Description
`challengeId`	string (24 hex)	yes	Returned by POST /auth/otp.

Request body (JSON)

Field	Type	Required	Rules / meaning
`code`	string	yes	4 to 8 digits.

Example request

```
{ "code": "1234" }
```

Success response · 200 OK

```
{ "verificationToken": "eyJhbGciOiJFZERTQSIs..." }
```

Specific errors

Status	code	When
400	OTP_INVALID	Wrong code. The body includes "attemptsLeft" (4, 3, 2, 1).
410	OTP_EXPIRED	More than 5 minutes, unknown challenge, or code already verified.
423	OTP_LOCKED	5 wrong attempts: the challenge is dead, ask for a new code.

Note. Attempts are counted atomically before comparing, so sending many guesses in parallel cannot exceed the limit.

Create the student account

POST `/api/v1/auth/register`

Step 3 of sign-up ("Profil"). Creates the user and the student profile for the phone proven by the verification token, and signs the student in immediately (returns tokens). The phone becomes the login identifier and the default Wave / Orange Money number for refunds. The student number is required and unique but not verified against a university registry.

Access Public (no token)

Rate limit 10 per hour per IP

Request body (JSON)

Field	Type	Required	Rules / meaning
`verificationToken`	string	yes	From POST /auth/otp/{id}/verify with purpose register. Single use.
`studentNumber`	string	yes	P + 6 digits. Normalized to upper case.
`firstName`	string	yes	1–60 letters (accents allowed), spaces, hyphens, apostrophes.
`lastName`	string	yes	Same rules as firstName.
`campus`	"campus1" | "campus2"	yes	
`onCampus`	boolean	yes	Lives in a campus residence. When false, village and room are ignored.
`village`	string | null	if onCampus	Must belong to the chosen campus: campus1 = Village A–F, campus2 = Village G–L.
`room`	string | null	if onCampus	As written on the door: 214, C12, 12B. Spaces removed, upper-cased.
`password`	string	yes	At least 8 characters including 1 digit, at most 128; very common passwords refused.
`termsVersion`	string	yes	Version of the terms the student accepted, e.g. "2026-01".
`device`	object	no	{ id?, platform?, appVersion? }: shown in the session list later.

Example request

```
{
  "verificationToken": "eyJhbGciOiJFZERTQSIs...",
  "studentNumber": "P241234",
  "firstName": "Khady",
  "lastName": "Mbaye",
  "campus": "campus2",
  "onCampus": true,
  "village": "Village H",
  "room": "C12",
  "password": "Bonjour2026",
  "termsVersion": "2026-01",
  "device": { "platform": "ios", "appVersion": "1.0.0" }
}
```

Success response · 201 Created

```
{
  "userId": "6ae5dc3870d01ba6d4ae2f15",
  "tokens": {
    "accessToken": "eyJhbGciOiJFZERTQSIsImtpZCI6ImRldi0xIn0...",
    "accessTokenExpiresAt": "2026-10-31T12:15:00.000Z",
    "refreshToken": "q3Vt8m1Jx0w...base64url (43 chars)",
    "refreshTokenExpiresAt": "2026-11-30T12:00:00.000Z"
  }
}
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	A field is invalid; the body says which one in "field" (studentNumber, firstName, village, room, password…).
409	PHONE_TAKEN	The phone got an account in the meantime.
409	STUDENT_NUMBER_TAKEN	The student number already has an account (an admin can reassign it after an ID check).
410	VERIFICATION_EXPIRED	Token expired (15 min), already used, or made for another purpose.

Sign in

POST `/api/v1/auth/login`

Exchanges an identifier and a password for tokens. The identifier can be the phone number (any common format) or the student number. An unknown account and a wrong password get different answers, so the app can say which one is wrong.

Access Public (no token)

Rate limit 30 per 10 minutes per IP · 10 per 10 minutes per identifier

Request body (JSON)

Field	Type	Required	Rules / meaning
identifier	string	yes	Phone ("77 000 00 02", "+221770000002") or student number ("P232055").
password	string	yes	
device	object	no	{ id?, platform?, appVersion? }

Example request

```
{ "identifier": "77 000 00 02", "password": "Campus2026" }
```

Success response · 200 OK

```
{ "userId": "6ae5dc1e15fb772d267ad55f", "tokens": { ...same as register... } }
```

Specific errors

Status	code	When
401	WRONG_PASSWORD	"Mot de passe incorrect.": the account exists but the password is wrong.
404	ACCOUNT_NOT_FOUND	"Aucun compte sur ce numéro.": no account for this phone or student number.
403	FORBIDDEN	Correct password but the account is suspended.
429	TOO_MANY_REQUESTS	Account temporarily locked after 5 wrong passwords (1, then 5, 15, 60 min; header Retry-After), or rate limit.

Note. After login, call GET /me to know which spaces (student, organizer) the account may open.

Get a new access token

POST `/api/v1/auth/refresh`

Access tokens live 15 minutes. When one expires (401), the app sends its refresh token here and receives a new pair. The refresh token is **rotated**: the old one stops working, so the app must store the new one every time.

Access Public (no token)

Rate limit 60 per 10 minutes per IP

Request body (JSON)

Field	Type	Required	Rules / meaning
`refreshToken`	string	yes	The latest refresh token received.

Example request

```
{ "refreshToken": "q3Vt8m1Jx0w..." }
```

Success response · 200 OK

```
{
  "accessToken": "eyJhbGciOiJFZERTQSIsImtpZCI6ImRldi0xIn0...",
  "accessTokenExpiresAt": "2026-10-31T12:15:00.000Z",
  "refreshToken": "q3Vt8m1Jx0w...base64url (43 chars)",
  "refreshTokenExpiresAt": "2026-11-30T12:00:00.000Z"
}
```

Specific errors

Status	code	When
401	UNAUTHENTICATED	Expired (30 days without use, 90 days max after login), revoked, or already used. The user must sign in again.

Note. Security: if an already-used refresh token is presented again (sign of theft), every session of that login is ended at once. The app should therefore never send two refresh calls in parallel with the same token.

Sign out on this device

POST `/api/v1/auth/logout`

Ends the session that owns this refresh token. The access token of that session stops working immediately.

Access Bearer token (any signed-in user)

Request body (JSON)

Field	Type	Required	Rules / meaning
`refreshToken`	string	yes	This device's refresh token. A token belonging to another user is ignored.

Example request

```
{ "refreshToken": "q3Vt8m1Jx0w..." }
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
401	UNAUTHENTICATED	Missing or invalid access token.

Sign out everywhere

POST `/api/v1/auth/logout-all`

Ends every session of the account on every device (lost phone, suspicion). Takes effect on the next request of each device.

Access Bearer token (any signed-in user)

Example request

```
(no body)
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
401	UNAUTHENTICATED	Missing or invalid access token.

Choose a new password (forgot password)

POST `/api/v1/auth/password/reset`

Last step of "mot de passe oublié": POST /auth/otp (purpose reset_password) → verify → this call. Sets the new password and ends every existing session. Also how accounts created by an admin (organizers) set their first password.

Access Public (no token)

Rate limit 10 per hour per IP

Request body (JSON)

Field	Type	Required	Rules / meaning
verificationToken	string	yes	From verify, with purpose reset_password. Single use.
newPassword	string	yes	Same rules as at sign-up.

Example request

```
{ "verificationToken": "eyJhbGciOiJFZERTQSIs...", "newPassword": "Nouveau2026" }
```

Success response · 204 No Content

```
(empty). The user then signs in with the new password.
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	Password rules not met (field newPassword).
410	VERIFICATION_EXPIRED	Token expired, used, or obtained for sign-up instead of reset.

Change my password

PUT `/api/v1/auth/password`

For a signed-in user who knows the current password. Keeps the current session and ends all the others.

Access Bearer token (any signed-in user)

Rate limit 10 per hour per IP

Request body (JSON)

Field	Type	Required	Rules / meaning
currentPassword	string	yes	
newPassword	string	yes	Same rules as at sign-up.

Example request

```
{ "currentPassword": "Bonjour2026", "newPassword": "Nouveau2026" }
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	Current password wrong (field currentPassword) or new one too weak (field newPassword).

My account

What the signed-in user can read and change about themselves. `GET /me` is the call that drives the app's `AuthContext`: name, profile and which spaces may be opened.

Method	Path	Goal
GET	/api/v1/me	Who am I, and which spaces can I open?
PATCH	/api/v1/me/student-profile	Update where I live
GET	/api/v1/me/notification-preferences	Read my notification settings
PUT	/api/v1/me/notification-preferences	Change my notification settings
PUT	/api/v1/me/email	Add or change my e-mail address
DELETE	/api/v1/me/email	Remove my e-mail address
PUT	/api/v1/me/location	Save my current position
DELETE	/api/v1/me/location	Forget my position

Who am I, and which spaces can I open?

GET **/api/v1/me**

Returns the profile shown in the Profil tab and the capabilities computed by the server: **spaces** contains "student" when the account has a student profile and "organizer" when it is an active member of at least one organization. The app uses it after login and on start-up to decide where to go; the server still checks rights on every call.

Access Bearer token (any signed-in user)

Example request

```
(no body)
```

Success response · 200 OK

```
{
  "id": "6ae5dc1e15fb772d267ad55f",
  "firstName": "Awa",
  "lastName": "Diop",
  "initials": "AD",
  "phoneMasked": "77 ••• •• 02",
  "platformRoles": [],
  "student": {
    "studentNumber": "P232055",
    "campus": "campus1",
    "onCampus": true,
    "village": "Village E",
    "room": "214",
    "ufr": null, "filiere": null, "niveau": null,
    "verified": false
  },
  "spaces": ["student", "organizer"],
  "organizations": [
    { "id": "6ae5dc1e15fb772d267ad55d", "name": "Amicale de Ziguinchor",
      "category": "Amicale", "level": "verifie", "roles": ["owner"] }
  ],
  "payoutPhoneMasked": "77 ••• •• 02",
  "email": { "address": "awa.diop@ugb.edu.sn", "verified": false },   // null if none
  "avatarUrl": null,                                                  // until photo upload ships
  "location": { "lat": 16.0617, "lng": -16.4236, "accuracyM": 12,
                "updatedAt": "2026-10-31T10:00:00.000Z" }           // null if never sent
}
```

Specific errors

Status	code	When
401	UNAUTHENTICATED	Missing, expired or revoked access token.

Note. student is null for organizer-only accounts. ufr / filiere / niveau are not collected yet (open question Q-13). email, avatarUrl and location are only ever returned to the user themself.

Update where I live

PATCH `/api/v1/me/student-profile`

Lets a student change residence: moved off campus, new village or new room. Student number, name and campus are not editable here (admin only).

Access Bearer token + student profile

Request body (JSON)

Field	Type	Required	Rules / meaning
onCampus	boolean	yes	False clears village and room.
village	string | null	if onCampus	Must belong to the student's campus.
room	string | null	if onCampus	214, C12, 12B…

Example request

```
{ "onCampus": true, "village": "Village E", "room": "108" }
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	Village not on this campus, or room format (field village / room).
403	FORBIDDEN	The account has no student profile.

Read my notification settings

GET `/api/v1/me/notification-preferences`

Which push notification categories the user wants, matching the inbox filters: Social, Cours, Paiements. All true by default.

Access Bearer token (any signed-in user)

Example request

```
(no body)
```

Success response · 200 OK

```
{ "social": true, "cours": true, "paiements": true }
```

Specific errors

Status	code	When
401	UNAUTHENTICATED	

Change my notification settings

PUT `/api/v1/me/notification-preferences`

Replaces the three switches. A category turned off still fills the in-app inbox but sends no push (once the notifications module ships).

Access Bearer token (any signed-in user)

Request body (JSON)

Field	Type	Required	Rules / meaning
`social`	boolean	yes	
`cours`	boolean	yes	
`paiements`	boolean	yes	

Example request

```
{ "social": false, "cours": true, "paiements": true }
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	A field is missing or not a boolean.

Add or change my e-mail address

PUT `/api/v1/me/email`

E-mail is optional and added after sign-up (Profil). It is stored lower-cased and **unverified**: verification by e-mail will come once an e-mail provider is chosen. It is not a login identifier. Changing it resets the verification.

Access Bearer token (any signed-in user)
Rate limit 10 per hour per user

Request body (JSON)

Field	Type	Required	Rules / meaning
`email`	string	yes	A valid address, up to 254 characters. Spaces around it are ignored.

Example request

```
{ "email": "Awa.Diop@ugb.edu.sn" }
```

Success response · 204 No Content

```
(empty). GET /me now returns { "address": "awa.diop@ugb.edu.sn", "verified": false }.
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	Not an e-mail address (field email).
409	EMAIL_TAKEN	Another account already uses this address (letter case ignored).

Remove my e-mail address

DELETE `/api/v1/me/email`

Deletes the address from the account; it becomes free for another account.

Access Bearer token (any signed-in user)

Example request

```
(no body)
```

Success response · 204 No Content

```
(empty)
```

Save my current position

PUT `/api/v1/me/location`

Stores the latitude and longitude the phone reports, for future features (nearby taxis, places, pick-up points). Stored as a GeoJSON point so "near me" searches can be added later. The position is personal data: it is **only returned to the user**, never to other students or organizers. The app should send it only after the user granted the location permission, and not more than every few minutes.

Access Bearer token (any signed-in user)

Rate limit 60 per hour per user

Request body (JSON)

Field	Type	Required	Rules / meaning
`lat`	number	yes	Latitude, -90 to 90 (UGB is about 16.06).
`lng`	number	yes	Longitude, -180 to 180 (UGB is about -16.42).
`accuracyM`	number	no	Accuracy radius reported by the phone, in metres (0 to 100 000).

Example request

```
{ "lat": 16.0617, "lng": -16.4236, "accuracyM": 12 }
```

Success response · 204 No Content

```
(empty). GET /me returns it in "location" with the server time of the update.
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	Missing coordinate, not a number, or out of range.

Forget my position

DELETE `/api/v1/me/location`

Erases the stored position (for example when the user turns location off in the app settings).

Access Bearer token (any signed-in user)

Example request

```
(no body)
```

Success response · 204 No Content

```
(empty)
```

Payments

One payment per purchase attempt (a taxi seat, a caravan ticket). **Only the server decides that money arrived**, from a signed provider webhook: the app opens `checkoutUrl`, then reads the payment until it is settled. Money sent back (refunds) and organizer claims are **payouts** to a Wave / Orange Money number. In development, Wave and Orange Money are simulated (`PAYMENTS_SIMULATOR=true`, refused in production).

Method	Path	Goal
`GET`	`/api/v1/payments/{reference}`	Read a payment
`GET`	`/api/v1/me/payouts`	Money sent to me
`GET`	`/api/v1/admin/payouts`	Payouts to follow up (finance)
`GET`	`/api/v1/dev/payment-simulator/{reference}`	Payment page (simulator, development only)
`POST`	`/api/v1/dev/payment-simulator/{reference}/{action}`	Simulate the provider's answer (development only)

Read a payment

GET **`/api/v1/payments/{reference}`**

After the payment page closes, the app polls this until the status is no longer pending. Other users' payments answer 404.

Access Bearer token (any signed-in user)

Path parameters

Name	Type	Required	Description
`reference`	string	yes	PAY_ + 20 characters, returned when the purchase started.

Example request

```
GET /api/v1/payments/PAY_4Q8M2KJ7XN1RTV5CW9EH
```

Success response · 200 OK

```
{
  "reference": "PAY_4Q8M2KJ7XN1RTV5CW9EH",
  "status": "pending",
      // created | pending | succeeded | failed | expired | cancelled
  "fulfillment": "pending",           // fulfilled | unfulfillable (paid too late: refunded)
  "amount": 750,
  "currency": "XOF",
  "method": "wave",
  "purpose": "taxi_seat",             // or caravan_booking
  "purposeRef": "6ae5f2305a0d54fc68a68549",
  "checkoutUrl": "http://192.168.1.151:3000/api/v1/dev/payment-simulator/PAY_4Q8M…",
  "expiresAt": "2026-10-31T14:20:00.000Z"
}
```

Money sent to me

GET **`/api/v1/me/payouts`**

Refunds (and claims, for organizers) sent to my Wave / Orange Money number, newest first.

Access Bearer token (any signed-in user)

Example request

```
GET /api/v1/me/payouts
```

Success response · 200 OK

```
{ "items": [ {
    "id": "6ae600f9425bab5a8336b232",
    "kind": "refund",                    // or organizer_claim
    "reason": "taxi_left",
    "amount": 500,
    "method": "wave",
    "status": "requested",               // processing | succeeded | failed | manual_required
    "phoneMasked": "77 ••• •• 02",
    "createdAt": "2026-10-31T14:53:45.584Z"
  } ] }
```

Payouts to follow up (finance)

GET `/api/v1/admin/payouts`

Back office: every payout, filterable by status. `manual_required` means the provider couldn't send it after 5 tries (or has no payout API): the finance team pays it by hand.

Access Bearer token + platform role **finance** (or admin)

Example request

```
GET /api/v1/admin/payouts?status=manual_required
```

Success response · 200 OK

```
{ "items": [ … ] }
```

Payment page (simulator, development only)

GET `/api/v1/dev/payment-simulator/{reference}`

Stands in for the Wave / Orange Money screens: an HTML page with the amount and buttons *Payer*, *Échec du paiement*, *Annuler* and *Payer (webhook envoyé 2 fois)*. Exists only when the simulator is on.

Access Public (no token) · development only

Path parameters

Name	Type	Required	Description
`reference`	string	yes	The payment reference.

Example request

```
GET /api/v1/dev/payment-simulator/PAY_4Q8M2KJ7XN1RTV5CW9EH   (opened in the phone's browser)
```

Success response · 200 OK

```
text/html: the simulated checkout page
```

Simulate the provider's answer (development only)

POST `/api/v1/dev/payment-simulator/{reference}/{action}`

Pressed from the simulator page. Builds a provider-style event, signs it, and sends it to the real webhook route, so the same code path as production runs (signature check, event inbox, dedupe, the purchase is fulfilled). Then redirects to the app's return link.

Access Public (no token) · development only

Path parameters

Name	Type	Required	Description
`reference`	string	yes	
`action`	pay | fail | cancel | duplicate	yes	duplicate = paid, webhook sent twice (tests the dedupe).

Example request

```
POST /api/v1/dev/payment-simulator/PAY_4Q8M2KJ7XN1RTV5CW9EH/pay
```

Success response · 303 See Other

```
Location: ugbcampus://paiement/retour?ref=PAY_…&status=succeeded   (or an HTML page without
    return link)
```

Notifications

The bell (in-app list, kept 90 days) and push notifications. Features notify on their events: ticket offered / accepted / declined, departure confirmed or cancelled, refund sent, someone joined your taxi, new chat message (push only). Push goes through Expo's push service to every phone of the account, unless its Social / Cours / Paiements switch is off. A new notification also arrives live on the WebSocket (`notification.created`).

Method	Path	Goal
`PUT`	`/api/v1/me/push-tokens`	Register this phone for push
`POST`	`/api/v1/me/push-tokens/unregister`	Stop push on this phone
`GET`	`/api/v1/notifications`	The bell list
`GET`	`/api/v1/notifications/unread-count`	The bell's dot
`POST`	`/api/v1/notifications/read`	Mark as read
`POST`	`/api/v1/admin/notifications`	Notify one account (back office)

Register this phone for push

PUT **`/api/v1/me/push-tokens`**

After sign-in, the app sends its Expo push token. A phone switching account moves to the new one.

Access Bearer token (any signed-in user)

Request body (JSON)

Field	Type	Required	Rules / meaning
`token`	string	yes	ExponentPushToken[…]
`platform`	ios | android | web	yes	

Example request

```
{ "token": "ExponentPushToken[xxxxxxxxxxxxxxxxxxxxxx]", "platform": "android" }
```

Success response · 204 No Content

```
(empty)
```

Stop push on this phone

POST **`/api/v1/me/push-tokens/unregister`**

On sign-out.

Access Bearer token (any signed-in user)

Request body (JSON)

Field	Type	Required	Rules / meaning
`token`	string	yes	

Example request

```
{ "token": "ExponentPushToken[xxxxxxxxxxxxxxxxxxxxxx]" }
```

Success response · 204 No Content

```
(empty)
```

The bell list

GET `/api/v1/notifications`

Newest first, filterable by category; `before` pages further back. `url` is the screen to open on tap.

Access Bearer token (any signed-in user)

Example request

```
GET /api/v1/notifications?category=paiements
```

Success response · 200 OK

```
{ "items": [ {
    "id": "6ae62c84a309c6c6c4b26e74",
    "category": "paiements",            // social | cours | paiements
    "type": "caravan.confirmed",
    "title": "Départ confirmé · UGB → Dakar",
    "body": "Montée le sam. 02 nov., 21:01 à UGB · Village E. Sois-y 15 min avant.",
    "icon": "bus-outline", "tone": "success",
    "url": "/billet/6ae620c90616e4e069d84afc",
    "read": false,
    "createdAt": "2026-10-31T17:59:32.973Z"
  } ], "hasMore": false }
```

The bell's dot

GET `/api/v1/notifications/unread-count`

Notifications not read yet.

Access Bearer token (any signed-in user)

Example request

```
GET /api/v1/notifications/unread-count
```

Success response · 200 OK

```
{ "count": 3 }
```

Mark as read

POST `/api/v1/notifications/read`

Some, or all (opening the notifications screen).

Access Bearer token (any signed-in user)

Request body (JSON)

Field	Type	Required	Rules / meaning
`ids`	ObjectId[]	one of	Up to 100.
`all`	boolean	one of	

Example request

```
{ "all": true }
```

Success response · 204 No Content

```
(empty)
```

Notify one account (back office)

POST `/api/v1/admin/notifications`

A message from the team, or a quick test of push on a phone.

Access Bearer token + platform role **admin**

Request body (JSON)

Field	Type	Required	Rules / meaning
`phone`	string	yes	
`title`	string ≤ 120	yes	
`body`	string ≤ 300	yes	
`category`	social | cours | paiements	yes	
`url`	app route	no	Default /notifications.

Example request

```
{ "phone": "77 000 00 02", "title": "Test de notification",
    "body": "Envoyé depuis le back-office", "category": "social" }
```

Success response · 202 Accepted

```
{ "queued": true }
```

Sunu Taxi

A student proposes a shared taxi; others pay their seat to join. The server sets the seat price (the fare split as if the taxi were full, rounded up to 50 FCFA) and the lock time (30 min before departure, or half-way for trips proposed less than an hour ahead). People are shown by first name + initial and a pseudonymous `key`, never a phone or an id. One active trip per student.

Method	Path	Goal
GET	/api/v1/taxi/trips	Open trips (the board)
GET	/api/v1/taxi/trips/{id}	One trip
GET	/api/v1/taxi/me	My Sunu Taxi standing
POST	/api/v1/taxi/trips	Propose a trip
POST	/api/v1/taxi/trips/{id}/join	Take a seat and pay
GET	/api/v1/taxi/trips/{id}/leave-quote	What leaving would refund
POST	/api/v1/taxi/trips/{id}/leave	Leave a trip
POST	/api/v1/taxi/trips/{id}/cancel	Cancel my trip (organizer)
POST	/api/v1/taxi/trips/{id}/handover	Hand the trip over ("Passer le relais")
POST	/api/v1/taxi/trips/{id}/check-in	"Je suis là"
PUT	/api/v1/taxi/trips/{id}/vehicle	Say which taxi it is
POST	/api/v1/taxi/trips/{id}/depart	"On est partis"
POST	/api/v1/taxi/trips/{id}/claim	Collect the paid seats

Open trips (the board)

GET **/api/v1/taxi/trips**

Open trips (until 15 min after their time), plus mine while their chat is open. `serverTime` lets the app correct a phone set to the wrong time.

Access Bearer token + student profile

Example request

```
GET /api/v1/taxi/trips
```

Success response · 200 OK

```
{ "serverTime": "2026-10-31T14:47:00.000Z", "items": [ {
    "id": "6ae5f2185a0d54fc68a68541",
    "from": "UGB · Village E", "to": "Saint-Louis · Gare routière",
    "departureAt": "2026-10-31T15:30:00.000Z",
    "lockAt": "2026-10-31T15:00:00.000Z",      // seats lock: leaving is no longer refunded
    "chatClosesAt": "2026-10-31T17:30:00.000Z",
    "seats": 4, "seatsTaken": 2, "fare": 3000, "seatPrice": 750,
    "status": "open",                           // open | departed | cancelled
    "organizer": { "key": "a1b2c3d4e5f60718", "name": "Moussa S.", "initials": "MS", "trips": 23,
        "lateCancels": 0 },
    "passengers": [ { "key": "a1b2…", "name": "Moussa S.", "initials": "MS", "role": "organizer",
        "checkedIn": false }, … ],
    "vehicle": null,                            // "Peugeot 406 jaune · DK-1234-A",
        people of the trip only
    "conversationId": null,                     // the trip chat, people of the trip only
    "me": null,                                 // { key, role, status, checkedIn, forfeited,
        payment }
    "claim": null                               // organizer after departure: { amount, payout }
  } ] }
```

One trip

GET `/api/v1/taxi/trips/{id}`

A trip with my place in it. Departed or cancelled trips are visible only to the people who were in them.

Access Bearer token + student profile

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/taxi/trips/6ae5f2185a0d54fc68a68541
```

Success response · 200 OK

```
{
  "id": "6ae5f2185a0d54fc68a68541",
  "from": "UGB · Village E", "to": "Saint-Louis · Gare routière",
  "departureAt": "2026-10-31T15:30:00.000Z",
  "lockAt": "2026-10-31T15:00:00.000Z",      // seats lock: leaving is no longer refunded
  "chatClosesAt": "2026-10-31T17:30:00.000Z",
  "seats": 4, "seatsTaken": 2, "fare": 3000, "seatPrice": 750,
  "status": "open",                           // open | departed | cancelled
  "organizer": { "key": "a1b2c3d4e5f60718", "name": "Moussa S.", "initials": "MS", "trips": 23,
      "lateCancels": 0 },
  "passengers": [ { "key": "a1b2…", "name": "Moussa S.", "initials": "MS", "role": "organizer",
      "checkedIn": false }, … ],
  "vehicle": null,                            // "Peugeot 406 jaune · DK-1234-A",
      people of the trip only
  "conversationId": null,                     // the trip chat, people of the trip only
  "me": null,                                 // { key, role, status, checkedIn, forfeited,
      payment }
  "claim": null                               // organizer after departure: { amount, payout }
}
```

My Sunu Taxi standing

GET `/api/v1/taxi/me`

My key, late cancellations over 30 days (3 suspend Sunu Taxi for 7 days), the suspension end, and my active trip.

Access Bearer token + student profile

Example request

```
GET /api/v1/taxi/me
```

Success response · 200 OK

```
{ "key": "ebb1145b59e9ec7f", "strikes": 0, "limit": 3, "suspendedUntil": null,
    "activeTripId": null }
```

Propose a trip

POST `/api/v1/taxi/trips`

The "Proposer un trajet" form. A retried request returns the same trip (200) instead of creating a second one.

Access Bearer token + student profile

Rate limit 10 per hour per student

Request body (JSON)

Field	Type	Required	Rules / meaning
`from`	string	yes	Free text, 2–80 characters.
`to`	string	yes	Free text, different from `from`.
`seats`	integer 2–4	yes	Seats in the taxi, organizer included.
`fare`	integer 500–20 000	yes	Whole fare, FCFA.
`leaveInMinutes`	integer	one of	Measured on the server clock (safe from a phone at the wrong time). 10 min to 7 days.
`departureAt`	ISO date	one of	Instead of leaveInMinutes.

Example request

```
{ "from": "UGB · Village E", "to": "Saint-Louis · Gare routière", "seats": 4, "fare": 3000,
    "leaveInMinutes": 45 }
```

Success response · 201 Created

```
the trip (as above)
```

Specific errors

Status	code	When
409	`ALREADY_IN_TRIP`	Already organizing or riding another trip (tripId in the body).
403	`TAXI_SUSPENDED`	3 late cancellations in 30 days.
422	`DEPARTURE_TOO_SOON`	Less than 10 minutes ahead.

Take a seat and pay

POST `/api/v1/taxi/trips/{id}/join`

Holds a seat for 10 minutes and opens the payment. Joined once the provider confirms. Calling again while the hold runs returns the same payment (200).

Access	Bearer token + student profile
Rate limit	30 per hour per student

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`method`	"wave" | "orange_money"	yes	How the student pays. In development both go through the payment simulator.
`returnUrl`	string	no	App deep link the payment page sends the student back to (ugbcampus://… or exp://… in development). Other schemes are refused.

Example request

```
{ "method": "wave", "returnUrl": "ugbcampus://paiement/retour" }
```

Success response · 201 Created

```
{ "trip": { … "me": { "status": "pending_payment", "payment": { "reference": "PAY_…", … } } },
    "payment": {
    "reference": "PAY_4Q8M2KJ7XN1RTV5CW9EH",
    "status": "pending",
        // created | pending | succeeded | failed | expired | cancelled
    "fulfillment": "pending",           // fulfilled | unfulfillable (paid too late: refunded)
    "amount": 750,
    "currency": "XOF",
    "method": "wave",
    "purpose": "taxi_seat",             // or caravan_booking
    "purposeRef": "6ae5f2305a0d54fc68a68549",
    "checkoutUrl": "http://192.168.1.151:3000/api/v1/dev/payment-simulator/PAY_4Q8M…",
    "expiresAt": "2026-10-31T14:20:00.000Z"
  } }
```

Specific errors

Status	code	When
409	`TRIP_FULL`	No seat left.
409	`ALREADY_IN_TRIP`	In another trip.
409	`TRIP_CLOSED`	Departed, cancelled or past.

What leaving would refund

GET `/api/v1/taxi/trips/{id}/leave-quote`

Shown before confirming "Quitter": 100 % before the lock, nothing after.

Access	Bearer token + student profile

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/taxi/trips/{id}/leave-quote
```

Success response · 200 OK

```
{ "refundAmount": 750, "locked": false }
```

Leave a trip

POST **/api/v1/taxi/trips/{id}/leave**

Before the lock: refunded to my Wave / Orange Money number, the seat reopens. After: no refund, the seat stays paid for the organizer (I can still read the chat). An unpaid seat is simply released.

Access Bearer token + student profile

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/taxi/trips/{id}/leave
```

Success response · 200 OK

```
{ "refundAmount": 750, "forfeited": false }
```

Specific errors

Status	code	When
409	ORGANIZER_CANNOT_LEAVE	The organizer hands over or cancels instead.

Cancel my trip (organizer)

POST **/api/v1/taxi/trips/{id}/cancel**

Everyone who paid is refunded 100 %. After the lock, with paid passengers, it counts as a late cancellation.

Access Bearer token + student profile · trip organizer

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/taxi/trips/{id}/cancel
```

Success response · 200 OK

```
{ "refunded": 2, "strike": true, "strikes": 1, "limit": 3, "suspendedUntil": null }
```

Hand the trip over ("Passer le relais")

POST `/api/v1/taxi/trips/{id}/handover`

The first passenger who joined organizes instead (their seat is refunded); the former organizer leaves. No penalty.

Access Bearer token + student profile · trip organizer

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/taxi/trips/{id}/handover
```

Success response · 200 OK

```
the trip
```

Specific errors

Status	code	When
409	NO_PASSENGER	Nobody has paid yet: cancel instead (free).

"Je suis là"

POST `/api/v1/taxi/trips/{id}/check-in`

At the meeting point. Once is enough; the chat shows "Awa est là □".

Access Bearer token + student profile · paid passenger

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/taxi/trips/{id}/check-in
```

Success response · 200 OK

```
the trip
```

Say which taxi it is

PUT `/api/v1/taxi/trips/{id}/vehicle`

Pinned in the chat for the trip's people ("Peugeot 406 jaune · DK-1234-A").

Access Bearer token + student profile · trip organizer

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`label`	string 2–60	yes	Model, colour, plate.

Example request

```
{ "label": "Peugeot 406 jaune · DK-1234-A" }
```

Success response · 200 OK

```
the trip
```

"On est partis"

POST `/api/v1/taxi/trips/{id}/depart`

From the lock until the chat closes. Unpaid holds are dropped; the organizer can then collect the money.

Access Bearer token + student profile · trip organizer

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/taxi/trips/{id}/depart
```

Success response · 200 OK

```
the trip, with "claim": { "amount": 1500, "payout": null }
```

Specific errors

Status	code	When
422	`TOO_EARLY_TO_DEPART`	Before the lock.

Collect the paid seats

POST `/api/v1/taxi/trips/{id}/claim`

After departure, the organizer receives paid seats × seat price (seats kept after a late leave included) on their own number. Once per trip.

Access Bearer token + student profile · trip organizer

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`method`	"wave" | "orange_money"	yes	How the student pays. In development both go through the payment simulator.

Example request

```
{ "method": "wave" }
```

Success response · 200 OK

```
{
  "id": "6ae600f9425bab5a8336b232",
  "kind": "organizer_claim",                    // or organizer_claim
  "reason": "taxi_claim",
  "amount": 500,
  "method": "wave",
  "status": "requested",              // processing | succeeded | failed | manual_required
  "phoneMasked": "77 ••• •• 02",
  "createdAt": "2026-10-31T14:53:45.584Z"
}
```

Specific errors

Status	code	When
409	TRIP_NOT_DEPARTED	Confirm the departure first.
422	NOTHING_TO_CLAIM	Nobody paid.

Chat

One chat for every feature (Sunu Taxi now, lost & found next). The feature that owns a conversation decides who is in it and who may write; messages are sent here and received live on the WebSocket `/api/v1/ws` (first frame `{type:'auth', accessToken}`, then `{type:'sub', topic:'conversation:<id>'}`). Non-members get 404.

Method	Path	Goal
GET	/api/v1/conversations	My conversations and their badges
GET	/api/v1/conversations/{id}	One conversation
GET	/api/v1/conversations/{id}/messages	Messages
POST	/api/v1/conversations/{id}/messages	Send a message
POST	/api/v1/conversations/{id}/read	Mark as read
POST	/api/v1/conversations/{id}/messages/{messageId}/report	Report a message
POST	/api/v1/conversations/{id}/messages/{messageId}/block-author	Block the author
GET	/api/v1/admin/message-reports	Reported messages

My conversations and their badges

GET `/api/v1/conversations`

Every conversation I'm in, newest activity first, with unread counts (others' messages after my read marker).

Access Bearer token (any signed-in user)

Example request

```
GET /api/v1/conversations?type=taxi_trip
```

Success response · 200 OK

```
{ "items": [ { "id": "…", "type": "taxi_trip", "contextRef": "<trip id>",
    "title": "UGB · Village E → Saint-Louis · Gare routière", "canWrite": true, "unread": 2,
    "lastMessage": { … } } ] }
```

One conversation

GET `/api/v1/conversations/{id}`

Members (by key and name), whether I can write, and the members I blocked (to hide their live messages too).

Access Bearer token (any signed-in user) · member

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/conversations/{id}
```

Success response · 200 OK


```
{ "id": "…", "type": "taxi_trip", "contextRef": "…", "title": "…", "status": "open",
    "canWrite": true, "writableUntil": "…",
  "me": { "key": "…", "name": "Awa D.", "initials": "AD", "role": "passenger" },
      "members": [ … ], "blockedKeys": [], "unread": 0 }
```


Messages

GET `/api/v1/conversations/{id}/messages`

Oldest first. Without parameters: the latest page. `before`: older ones. `after`: what was missed while offline. Messages of people I blocked are left out.

Access Bearer token (any signed-in user) · member

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	

Example request


```
GET /api/v1/conversations/{id}/messages?after=6ae60d3fdf55979c83a1ec08
```


Success response · 200 OK


```
{ "items": [ {
    "id": "6ae60d3fdf55979c83a1ec08",
    "conversationId": "6ae60c11df55979c83a1ebf0",
    "kind": "text",                    // text | quick_reply | system
    "body": "J'arrive dans 5 min",
    "author": { "key": "ebb1145b59e9ec7f", "name": "Awa D.", "initials": "AD",
        "role": "passenger" },   // null for system lines
    "clientMessageId": "lq3k9x-4f8a2b1c9d",
    "createdAt": "2026-10-31T15:12:04.000Z"
  } ], "hasMore": false }
```


Send a message

POST `/api/v1/conversations/{id}/messages`

A retry with the same `clientMessageId` returns the first message (200) instead of posting twice. Other members get a push.

Access Bearer token (any signed-in user) · member allowed to write

Rate limit 20 per minute per conversation

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`body`	string 1–500	yes	
`kind`	text | quick_reply	no	Default text.
`clientMessageId`	string 8–64	yes	Made by the app per message.

Example request


```
{ "body": "J'arrive dans 5 min", "kind": "quick_reply", "clientMessageId": "lq3k9x-4f8a2b1c9d" }
```

Success response · 201 Created

```
{
  "id": "6ae60d3fdf55979c83a1ec08",
  "conversationId": "6ae60c11df55979c83a1ebf0",
  "kind": "text",                    // text | quick_reply | system
  "body": "J'arrive dans 5 min",
  "author": { "key": "ebb1145b59e9ec7f", "name": "Awa D.", "initials": "AD",
      "role": "passenger" },   // null for system lines
  "clientMessageId": "lq3k9x-4f8a2b1c9d",
  "createdAt": "2026-10-31T15:12:04.000Z"
}
```

Specific errors

Status	code	When
403	CHAT_READ_ONLY	Trip over, cancelled, or I left after the lock (read only).

Mark as read

POST **/api/v1/conversations/{id}/read**

Moves my read marker forward (never back). Clears the badge.

Access Bearer token (any signed-in user) · member

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
lastReadMessageId	ObjectId	yes	Newest message seen.

Example request

```
{ "lastReadMessageId": "6ae60d3fdf55979c83a1ec08" }
```

Success response · 204 No Content

```
(empty)
```

Report a message

POST **/api/v1/conversations/{id}/messages/{messageId}/report**

A copy goes to the team for review; the author isn't told who reported. Twice is a no-op.

Access Bearer token (any signed-in user) · member

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	
messageId	ObjectId	yes	Someone else's message.

Request body (JSON)

Field	Type	Required	Rules / meaning
reason	string ≤ 300	no	

Example request

```
{ "reason": "insulte" }
```

Success response · 204 No Content

```
(empty)
```

Block the author

POST `/api/v1/conversations/{id}/messages/{messageId}/block-author`

Hides this person's messages from me in every conversation of every feature, now and later.

Access Bearer token (any signed-in user) · member

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	
`messageId`	ObjectId	yes	

Example request

```
POST …/block-author
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
422	`OWN_MESSAGE`	My own message.
422	`SYSTEM_MESSAGE`	A line from the app.

Reported messages

GET `/api/v1/admin/message-reports`

Back office: reports waiting for review, oldest first.

Access Bearer token + platform role **admin**

Example request

```
GET /api/v1/admin/message-reports
```

Success response · 200 OK

```
{ "items": [ { "id": "…", "conversationId": "…", "conversationType": "taxi_trip",
    "messageId": "…", "body": "Message déplacé", "reason": "insulte", "createdAt": "…" } ] }
```

Caravans

Departures sold seat by seat. A seat is held 10 minutes while paying; the ticket is issued when the payment is confirmed. A departure is **confirmed** once paid seats reach its minimum; if not by `confirmBy`, it is cancelled and everyone refunded 100 %. One purchase per student per departure (still counted after giving the ticket away).

Method	Path	Goal
GET	/api/v1/caravans/departures	Departures to book
GET	/api/v1/caravans/departures/{id}	One departure
GET	/api/v1/caravans/departures/{id}/seats	Seat map
POST	/api/v1/caravans/departures/{id}/bookings	Book a seat and pay
GET	/api/v1/caravans/bookings/{id}	A booking (after paying)
DELETE	/api/v1/caravans/bookings/{id}	Release a seat I'm paying for

Departures to book

GET **/api/v1/caravans/departures**

Published and confirmed departures still to come, soonest first. Searching by stop is done by the app.

Access Bearer token + student profile

Example request

```
GET /api/v1/caravans/departures?direction=aller
```

Success response · 200 OK

```
{ "items": [ {
    "id": "6ae620d6e78b1b86ccfdf1cb",
    "code": "UGQ4KX2M",
    "status": "published",
        // draft | published | confirmed | cancelled | departed | completed
    "organizer": { "id": "…", "slug": "khoule", "name": "Khoulé & Frère",
        "category": "Entreprise", "level": "confirme", "rating": 4.2, "trips": 128,
        "logoUrl": null },
    "direction": "aller", "from": "UGB", "to": "Dakar",
    "departureAt": "2026-11-02T21:01:00.000Z",
    "pickups":  [ { "stopId": "p1", "place": "UGB · Village E",
        "at": "2026-11-02T21:01:00.000Z" }, … ],
    "dropoffs": [ { "stopId": "d1", "place": "Louga · Station Total",
        "at": "2026-11-02T23:11:00.000Z" }, … ],
    "price": 3500, "totalSeats": 64, "seatsFree": 63, "seatsBooked": 1,
    "minPassengers": 40, "confirmBy": "2026-11-01T21:01:00.000Z", "confirmed": false,
        "bookingOpen": true,
    "vehicle": { "label": "Bus 64 places", "plate": null },
        // plate shown to ticket holders only
    "myTicketId": null, "purchasedHere": false, "myBooking": null
  } ] }
```

One departure

GET **/api/v1/caravans/departures/{id}**

With my ticket, or my seat being paid (`myBooking`).

Access Bearer token + student profile

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/caravans/departures/{id}
```

Success response · 200 OK

```
{
  "id": "6ae620d6e78b1b86ccfdf1cb",
  "code": "UGQ4KX2M",
  "status": "published",
      // draft | published | confirmed | cancelled | departed | completed
  "organizer": { "id": "…", "slug": "khoule", "name": "Khoulé & Frère", "category": "Entreprise",
      "level": "confirme", "rating": 4.2, "trips": 128, "logoUrl": null },
  "direction": "aller", "from": "UGB", "to": "Dakar",
  "departureAt": "2026-11-02T21:01:00.000Z",
  "pickups":  [ { "stopId": "p1", "place": "UGB · Village E", "at": "2026-11-02T21:01:00.000Z" },
      … ],
  "dropoffs": [ { "stopId": "d1", "place": "Louga · Station Total",
      "at": "2026-11-02T23:11:00.000Z" }, … ],
  "price": 3500, "totalSeats": 64, "seatsFree": 63, "seatsBooked": 1,
  "minPassengers": 40, "confirmBy": "2026-11-01T21:01:00.000Z", "confirmed": false,
      "bookingOpen": true,
  "vehicle": { "label": "Bus 64 places", "plate": null },
      // plate shown to ticket holders only
  "myTicketId": null, "purchasedHere": false, "myBooking": null
}
```

Seat map

GET **/api/v1/caravans/departures/{id}/seats**

Taken seats only, never who sits where. `suggested`: what "Choisis pour moi" picks (front-most window seat).

Access Bearer token + student profile

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/caravans/departures/{id}/seats
```

Success response · 200 OK

```
{ "capacity": 64, "seatsPerRow": 4, "blocked": [], "taken": [1, 4, 6], "suggested": 5 }
```

Book a seat and pay

POST **/api/v1/caravans/departures/{id}/bookings**

Holds the seat 10 minutes and opens the payment. Coming back while it runs returns the same payment (200).

Access Bearer token + student profile

Rate limit 20 per hour per student

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
seat	integer	no	Omit to let the server choose.
pickupStopId	string	yes	"p1", "p2"…
dropoffStopId	string	yes	"d1"…, after the pickup on the route.
method	"wave" | "orange_money"	yes	How the student pays. In development both go through the payment simulator.
returnUrl	string	no	App deep link the payment page sends the student back to (ugbcampus://… or exp://… in development). Other schemes are refused.

Example request

```
{ "seat": 6, "pickupStopId": "p1", "dropoffStopId": "d7", "method": "wave",
    "returnUrl": "ugbcampus://paiement/retour" }
```

Success response · 201 Created

```
{ "booking": { "id": "…", "departureId": "…", "status": "held", "seat": 6, "holdExpiresAt": "…",
    "ticketId": null, "payment": { … } },
  "payment": { "reference": "PAY_…", "checkoutUrl": "…", … } }
```

Specific errors

Status	code	When
409	SEAT_TAKEN	Taken a moment ago: pick another.
409	NO_SEATS_LEFT	Full.
409	ALREADY_BOOKED	Already bought for this departure.
409	BOOKING_CLOSED	Bookings closed (30 min before by default).

A booking (after paying)

GET `/api/v1/caravans/bookings/{id}`

`confirmed` with a `ticketId` once paid; `expired` if the 10 minutes ran out.

Access Bearer token + student profile · own

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/caravans/bookings/{id}
```

Success response · 200 OK

```
{ "id": "…", "departureId": "…", "status": "confirmed", "seat": 6, "holdExpiresAt": "…",
    "ticketId": "…", "payment": { "reference": "PAY_…", "status": "succeeded",
    "checkoutUrl": null } }
```

Release a seat I'm paying for

DELETE `/api/v1/caravans/bookings/{id}`

"Libérer la place": the seat goes back to others at once.

Access Bearer token + student profile · own

Path parameters

Name	Type	Required	Description
`id`	ObjectId (24 hex)	yes	

Example request

```
DELETE /api/v1/caravans/bookings/{id}
```

Success response · 204 No Content

```
(empty)
```

Tickets

Tickets are generic: a `kind` (caravan seats now, other events later), an `event` card and kind-specific `details`. The QR code is **signed by the server** (no personal data inside); a transfer changes it, so screenshots of a ticket given away are refused. One valid ticket per person per event.

Method	Path	Goal
GET	/api/v1/tickets	My tickets
GET	/api/v1/tickets/{id}	One ticket, with its QR code
POST	/api/v1/tickets/{id}/cancel	Cancel my ticket
POST	/api/v1/tickets/{id}/transfers	Offer my ticket to a friend
GET	/api/v1/ticket-transfers	Offers waiting for an answer
POST	/api/v1/ticket-transfers/{id}/accept	Accept a ticket
POST	/api/v1/ticket-transfers/{id}/decline	Decline a ticket
POST	/api/v1/ticket-transfers/{id}/cancel	Withdraw my offer

My tickets

GET `/api/v1/tickets`

Tickets I hold (any status) and the ones I gave away (status `transferred`), newest first.

Access Bearer token (any signed-in user)

Example request

```
GET /api/v1/tickets
```

Success response · 200 OK

```
{ "items": [ … ] }
```

One ticket, with its QR code

GET `/api/v1/tickets/{id}`

Everything the ticket screen shows: QR payload, what cancelling would cost now, transfer deadline and pending offer.

Access Bearer token (any signed-in user) · holder (or former holder, read-only)

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/tickets/{id}
```

Success response · 200 OK

```
{
  "id": "6ae620c90616e4e069d84afc",
  "kind": "caravan_seat",
  "code": "NVBHQT",                    // typed by the scanner if the QR can't be shown
  "status": "active",                  // active | cancelled | transferred (I gave it away)
  "seat": 1, "price": 3500, "paidWith": "wave",
  "event": { "id": "…", "kind": "caravan_seat", "title": "UGB → Ziguinchor", "startsAt": "…",
      "status": "published",
            "organizer": { "id": "…", "name": "Globe One", "category": "Entreprise",
                "level": "confirme", "rating": 4.0, "trips": 86 } },
  "details": { "departureId": "…", "pickup": { "stopId": "p1", "place": "UGB · Village E",
      "at": "…" },
              "dropoff": { … }, "vehicle": { "label": "Bus 64 places", "plate": "DK-7710-AZ" },
              "confirmed": false, "minPassengers": 40, "seatsBooked": 1, "confirmBy": "…" },
  "qr": "eyJhbGciOiJFZERTQSIsImtp…",
      // signed; render as a QR code (holder of an active ticket only)
  "admittedAt": null,
  "cancellation": null,
  "cancellationQuote": { "allowed": true, "fee": 0, "refund": 3500,
      "freeUntil": "2026-11-01T21:24:00.000Z" },
  "transferDeadline": "2026-11-02T19:24:00.000Z",
  "transfer": null                     // pending offer, or the accepted one for the former holder
}
```

Cancel my ticket

POST `/api/v1/tickets/{id}/cancel`

Caravans: free until 24 h before departure, then 10 %; refused once boarded or after departure. The refund goes to the holder's number.

Access Bearer token (any signed-in user) · holder

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/tickets/{id}/cancel
```

Success response · 200 OK

```
{ "refundAmount": 3150, "fee": 350 }
```

Specific errors

Status	code	When
422	CANCELLATION_REFUSED	Boarded, departed, or the departure was cancelled (refunded automatically).

Offer my ticket to a friend

POST `/api/v1/tickets/{id}/transfers`

The friend (a registered student) must accept in their app; until then the ticket stays mine. Expires after 24 h or at the cutoff (caravans: departure − 2 h).

Access Bearer token (any signed-in user) · holder

Rate limit 10 per hour

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
recipientPhone	string	yes	The friend's phone.

Example request

```
{ "recipientPhone": "77 123 45 67" }
```

Success response · 201 Created

```
{ "id": "…", "ticketId": "…", "status": "pending", "expiresAt": "…",
    "from": { "name": "Awa D." }, "to": { "name": "Moussa S.", "phoneMasked": "77 ••• •• 67" } }
```

Specific errors

Status	code	When
422	RECIPIENT_NOT_FOUND	No student account on that number.
422	RECIPIENT_HAS_TICKET	They already hold one.
409	TRANSFER_PENDING	One offer at a time.
422	TRANSFER_CLOSED	Past the cutoff.

Offers waiting for an answer

GET `/api/v1/ticket-transfers`

`incoming`: tickets friends want to give me ("Billets reçus"). `outgoing`: my offers.

Access Bearer token (any signed-in user)

Example request

```
GET /api/v1/ticket-transfers?direction=incoming
```

Success response · 200 OK

```
{ "items": [ { "id": "…", "status": "pending", "from": { "name": "Awa D." }, "seat": 1,
    "event": { "title": "UGB → Ziguinchor", … }, "details": { … } } ] }
```

Accept a ticket

POST `/api/v1/ticket-transfers/{id}/accept`

The ticket becomes mine (new QR); the sender's QR stops working.

Access Bearer token (any signed-in user) · recipient

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/ticket-transfers/{id}/accept
```

Success response · 200 OK

```
{ "status": "accepted", "ticketId": "6ae620c90616e4e069d84afc" }
```

Specific errors

Status	code	When
410	TRANSFER_EXPIRED	Withdrawn, expired, or the ticket changed meanwhile.

Decline a ticket

POST `/api/v1/ticket-transfers/{id}/decline`

The ticket stays with the sender.

Access Bearer token (any signed-in user) · recipient

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/ticket-transfers/{id}/decline
```

Success response · 200 OK

```
{ "status": "declined", "ticketId": null }
```

Withdraw my offer

POST `/api/v1/ticket-transfers/{id}/cancel`

Before the friend answers.

Access Bearer token (any signed-in user) · sender

Path parameters

Name	Type	Required	Description
id	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/ticket-transfers/{id}/cancel
```

Success response · 204 No Content

```
(empty)
```

Organizer space

Routes under `/api/v1/org/{orgId}` serve the organizer space of the app. Access needs an **active membership** of that organization with the right role. Roles: **owner** (everything, including the team), **manager** (departures, menus, orders), **scanner** (boarding only), **staff** (restaurant orders). For anyone else, or for a suspended organization, the answer is **404**, so the API never confirms that an organization exists to people outside it.

Method	Path	Goal
GET	/api/v1/org/{orgId}	My organization and my role in it
PATCH	/api/v1/org/{orgId}	Update my organization's contact e-mail
GET	/api/v1/org/{orgId}/members	List my team
PUT	/api/v1/org/{orgId}/members	Add a team member or change their roles
DELETE	/api/v1/org/{orgId}/members/{userId}	Remove a team member

My organization and my role in it

GET **/api/v1/org/{orgId}**

Header of the organizer space ("Compte" tab): the organization card (name, category, e-mail, logo, verification level, rating, trips) and the caller's roles, so the app can hide actions the person may not take.

Access Bearer token + any role in the organization

Path parameters

Name	Type	Required	Description
orgId	string (24 hex)	yes	From GET /me → organizations[].id.

Example request

```
(no body)
```

Success response · 200 OK

```
{
  "organization": {
    "id": "6ae5dc1e15fb772d267ad55d",
    "slug": "amicale-zig",
    "name": "Amicale de Ziguinchor",
    "category": "Amicale",
    "capabilities": ["caravans"],
    "level": "verifie",
    "rating": 4.7,
    "trips": 5,
    "status": "active",
    "hasPayoutAccount": true,
    "email": "amicale.ziguinchor@ugb.edu.sn",
    "logoUrl": null
  },
  "myRoles": ["owner"]
}
```

Specific errors

Status	code	When
404	NOT_FOUND	Not a member, membership revoked, or organization suspended.

Update my organization's contact e-mail

PATCH `/api/v1/org/{orgId}`

Lets the owner add, change or remove the organization's contact e-mail after its registration. The logo will be editable here once photo upload ships.

Access Bearer token + role **owner**

Path parameters

Name	Type	Required	Description
`orgId`	string (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`email`	string | null	yes	Contact address, or null to remove it. Stored lower-cased.

Example request

```
{ "email": "commandes@chezfatou.sn" }
```

Success response · 200 OK

```
The organization (same shape as in GET /org/{orgId}).
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	Not an e-mail address.
404	NOT_FOUND	Caller is not owner of this organization.

List my team

GET `/api/v1/org/{orgId}/members`

"Mon équipe": who works for the organization and with which roles (drivers' helpers who scan tickets, managers…). Phone numbers are not returned.

Access Bearer token + role **owner**

Path parameters

Name	Type	Required	Description
`orgId`	string (24 hex)	yes	

Example request

```
(no body)
```

Success response · 200 OK

```
[
  { "userId": "6ae5…562", "roles": ["owner"], "firstName": "Modou", "lastName": "Khoulé" },
  { "userId": "6ae5…571", "roles": ["scanner"], "firstName": "Cheikh", "lastName": "Fall" }
]
```

Specific errors

Status	code	When
404	NOT_FOUND	Caller is not owner of this organization.

Add a team member or change their roles

PUT `/api/v1/org/{orgId}/members`

The owner gives roles to an existing UGB Campus account, found by phone number (for example a ticket scanner for the bus door). Calling it again for the same person replaces the roles and reactivates a removed member.

Access Bearer token + role **owner**

Path parameters

Name	Type	Required	Description
`orgId`	string (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`phone`	string	yes	Senegalese mobile, 9 digits starting with 70, 71, 75, 76, 77 or 78. Spaces and the +221 / 00221 prefix are accepted: "77 123 45 67", "+221771234567".
`roles`	array of "manager" | "scanner" | "staff"	yes	1 to 4 roles. "owner" is refused here: only an admin appoints owners.

Example request

```
{ "phone": "77 000 00 03", "roles": ["scanner"] }
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
403	FORBIDDEN	Tried to grant "owner".
404	NOT_FOUND	Caller is not owner of this organization.
422	USER_NOT_FOUND	No account with this phone: the person must sign up first (or an admin creates the account).

Remove a team member

DELETE `/api/v1/org/{orgId}/members/{userId}`

Revokes a member's access to the organizer space. Effective on their next request. The membership is kept as revoked (history).

Access Bearer token + role **owner**

Path parameters

Name	Type	Required	Description
`orgId`	string (24 hex)	yes	
`userId`	string (24 hex)	yes	From GET /org/{orgId}/members.

Example request

```
(no body)
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
404	NOT_FOUND	Not owner, or this person is not an active member.

Organizer: departures and boarding

The organizer app is for the day of the trip: "Mes activités", the passenger list stop by stop, scanning, and collecting the money. Creating vehicles and departures belongs to the web platform; until it exists, Swagger is the back office. Another organization's departure answers 404. Scans are live on every phone of the organization (WebSocket topic `event:<departureId>`).

Method	Path	Goal
GET	/api/v1/org/{orgId}/departures	"Mes activités"
GET	/api/v1/org/{orgId}/departures/{id}	One of my departures
GET	/api/v1/org/{orgId}/departures/{id}/manifest	Passenger list
GET	/api/v1/org/{orgId}/departures/{id}/offline-pack	Take the list offline
POST	/api/v1/org/{orgId}/admissions	Scan a ticket ("Faire monter")
POST	/api/v1/org/{orgId}/admissions/batch	Send scans made offline
POST	/api/v1/org/{orgId}/tickets/{ticketId}/admission/void	"Faire descendre"
POST	/api/v1/org/{orgId}/departures/{id}/stops/{stopId}/leave	"Le bus repart de cet arrêt"
POST	/api/v1/org/{orgId}/departures/{id}/claim	Collect the money ("Récupérer")
GET	/api/v1/org/{orgId}/vehicles	My vehicles
POST	/api/v1/org/{orgId}/vehicles	Register a vehicle
POST	/api/v1/org/{orgId}/departures	Create a departure (draft)
PUT	/api/v1/org/{orgId}/departures/{id}	Edit a draft
POST	/api/v1/org/{orgId}/departures/{id}/publish	Publish (open bookings)
POST	/api/v1/org/{orgId}/departures/{id}/cancel	Cancel a departure

"Mes activités"

GET **/api/v1/org/{orgId}/departures**

Upcoming (drafts included) or past departures, with booked / on-board counts.

Access Bearer token + role **owner**, **manager** or **scanner** in the organization

Path parameters

Name	Type	Required	Description
orgId	ObjectId	yes	The organization (the caller must be a member, else 404).

Example request

```
GET /api/v1/org/{orgId}/departures?scope=upcoming
```

Success response · 200 OK

```
{ "items": [ {
    … same fields as a student's departure …,
    "vehicle": { "label": "Bus 64 places", "plate": "DK-5502-BF" },
    "stopsLeft": [ { "stopId": "p1", "at": "2026-10-31T19:10:00.000Z" } ],
    "counts": { "booked": 3, "admitted": 1 },
    "cancellation": null,
    "claim": { "amount": 10500, "payout": null }     // after departure only
  } ] }
```

One of my departures

GET `/api/v1/org/{orgId}/departures/{id}`

Same card, for one departure.

Access Bearer token + role **owner**, **manager** or **scanner** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).
`id`	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/org/{orgId}/departures/{id}
```

Success response · 200 OK

```
{
  … same fields as a student's departure …,
  "vehicle": { "label": "Bus 64 places", "plate": "DK-5502-BF" },
  "stopsLeft": [ { "stopId": "p1", "at": "2026-10-31T19:10:00.000Z" } ],
  "counts": { "booked": 3, "admitted": 1 },
  "cancellation": null,
  "claim": { "amount": 10500, "payout": null }     // after departure only
}
```

Passenger list

GET `/api/v1/org/{orgId}/departures/{id}/manifest`

Every valid ticket with the holder's full name (checked against the student card) and phone (to call a late passenger), stops and boarding state. Every read is audited.

Access Bearer token + role **owner**, **manager** or **scanner** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).
`id`	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/org/{orgId}/departures/{id}/manifest
```

Success response · 200 OK

```
{ "departure": { … },
  "passengers": [ { "ticketId": "…", "code": "7KQ2MX", "seat": 1, "name": "Moussa Sow",
      "phone": "+221 77 123 45 67",
                  "pickupStopId": "p1", "dropoffStopId": "d2", "admitted": false,
                      "admittedAt": null, "transferred": false } ] }
```

Take the list offline

GET `/api/v1/org/{orgId}/departures/{id}/offline-pack`

Downloaded before leaving town: the passenger list, each ticket's QR version, the typed codes, and the public key that checks QR signatures. The phone then scans without network and sends the scans later (admissions/batch).

Access Bearer token + role **owner**, **manager** or **scanner** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).
`id`	ObjectId (24 hex)	yes	

Example request

```
GET /api/v1/org/{orgId}/departures/{id}/offline-pack
```

Success response · 200 OK

```
{ "generatedAt": "…", "key": { "kid": "dev-1", "alg": "EdDSA", "x": "<Ed25519 public key,
    base64url>" },
  "versions": { "<ticketId>": 1 }, "codes": { "7KQ2MX": "<ticketId>" }, "manifest": { … } }
```

Scan a ticket ("Faire monter")

POST `/api/v1/org/{orgId}/admissions`

A QR code, a typed code, or a ticket ticked in the list. Every scan is recorded; a ticket is boarded only once even if two phones scan it together. A retried scan (same clientScanId) gives the first answer again.

Access Bearer token + role **owner**, **manager** or **scanner** in the organization

Rate limit typed codes: 30 per minute per scanner

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).

Request body (JSON)

Field	Type	Required	Rules / meaning
`qr | code | ticketId`	string	one of	The QR payload, the 6-character code, or a ticket id (manual).
`eventId`	ObjectId	no	The departure the scanner is on: a ticket for another one is refused.
`locationId`	string	no	Stop where the bus is (default: first stop not left).
`clientScanId`	string 8–64	no	Made by the phone; makes the scan idempotent.
`deviceId, scannedAt`	string, ISO date	no	For the audit trail / offline scans.

Example request

```
{ "qr": "eyJhbGciOiJFZERTQSIsImtp…", "eventId": "6ae620d6e78b1b86ccfdf1cb",
    "clientScanId": "lq3m2a-9c1d7e" }
```

Success response · 200 OK

```
{
  "result": "admitted",              // admitted | duplicate | rejected
  "tone": "success",                 // success | warning | danger: the colour of the screen
  "title": "Moussa Sow · place 1",
  "text": "Descend à Dakar · Colobane. Vérifie le nom sur sa carte étudiant.",
  "reason": null,                    // rejected/duplicate: bad_signature, stale_version,
      unknown, wrong_org,
                                     // wrong_event, cancelled, too_early, too_late,
                                         already_admitted
  "eventId": "6ae620d6e78b1b86ccfdf1cb",
  "ticket": { "id": "…", "code": "7KQ2MX", "seat": 1 },
  "holder": { "name": "Moussa Sow" }
}
```

Send scans made offline

POST `/api/v1/org/{orgId}/admissions/batch`

Up to 200 scans queued without network, each with its clientScanId and scannedAt. The server stays the judge: if another phone boarded the same person first, the result is `duplicate`.

Access Bearer token + role **owner**, **manager** or **scanner** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).

Request body (JSON)

Field	Type	Required	Rules / meaning
`items`	array (1–200)	yes	Same fields as a single scan.

Example request

```
{ "items": [ { "qr": "…", "eventId": "…", "clientScanId": "off-lq3m-1",
    "scannedAt": "2026-10-31T19:02:11.000Z" } ] }
```

Success response · 200 OK

```
{ "items": [ { "result": "admitted", … }, { "result": "duplicate", "title": "Déjà à bord", … } ] }
```

"Faire descendre"

POST `/api/v1/org/{orgId}/tickets/{ticketId}/admission/void`

Undoes a boarding (wrong person, scanned by mistake). Kept in the log, marked void.

Access Bearer token + role **owner**, **manager** or **scanner** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).
`ticketId`	ObjectId	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`reason`	string ≤ 200	no	

Example request

```
{ "reason": "scanné par erreur" }
```

Success response · 204 No Content

```
(empty)
```

"Le bus repart de cet arrêt"

POST `/api/v1/org/{orgId}/departures/{id}/stops/{stopId}/leave`

Whoever was due at this stop and wasn't scanned is absent. Leaving the first pickup marks the departure departed.

Access Bearer token + role **owner**, **manager** or **scanner** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).
`id`	ObjectId (24 hex)	yes	
`stopId`	string	yes	A pickup: p1, p2…

Example request

```
POST /api/v1/org/{orgId}/departures/{id}/stops/p1/leave
```

Success response · 200 OK

```
{ "absent": [ { "ticketId": "…", "seat": 4 } ] }
```

Collect the money ("Récupérer")

POST `/api/v1/org/{orgId}/departures/{id}/claim`

After departure: valid tickets + fees kept on late cancellations, paid to the **organization's** payout number. Once per departure.

Access Bearer token + role **owner** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).
`id`	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`method`	"wave" | "orange_money"	yes	How the student pays. In development both go through the payment simulator.

Example request

```
{ "method": "orange_money" }
```

Success response · 200 OK

```
{
  "id": "6ae600f9425bab5a8336b232",
  "kind": "organizer_claim",                    // or organizer_claim
  "reason": "event_claim",
  "amount": 10500,
  "method": "wave",
  "status": "requested",              // processing | succeeded | failed | manual_required
  "phoneMasked": "77 ••• •• 02",
  "createdAt": "2026-10-31T14:53:45.584Z"
}
```

Specific errors

Status	code	When
409	DEPARTURE_NOT_DEPARTED	The bus hasn't left.
422	NO_PAYOUT_ACCOUNT	No payout number for the organization (admin sets it).
422	NOTHING_TO_CLAIM	No paid ticket.

My vehicles

GET `/api/v1/org/{orgId}/vehicles`

Active vehicles of the organization.

Access Bearer token + role **owner** or **manager** in the organization

Path parameters

Name	Type	Required	Description
orgId	ObjectId	yes	The organization (the caller must be a member, else 404).

Example request

```
GET /api/v1/org/{orgId}/vehicles
```

Success response · 200 OK

```
{ "items": [ { "id": "…", "label": "Bus 64 places", "plate": "DK-4821-BF", "capacity": 64,
    "seatsPerRow": 4, "blockedSeats": [] } ] }
```

Register a vehicle

POST `/api/v1/org/{orgId}/vehicles`

Its capacity is the most ever sold on a departure. Plates are unique.

Access Bearer token + role **owner** in the organization

Path parameters

Name	Type	Required	Description
orgId	ObjectId	yes	The organization (the caller must be a member, else 404).

Request body (JSON)

Field	Type	Required	Rules / meaning
`label`	string	yes	"Bus 64 places"
`plate`	string	yes	Normalized: DK-4821-BF
`capacity`	integer 4–90	yes	
`seatsPerRow`	integer 2–6	no	Default 4.
`blockedSeats`	integer[]	no	Never sold.

Example request

```
{ "label": "Bus 64 places", "plate": "DK 4821 BF", "capacity": 64 }
```

Success response · 201 Created

```
{ "id": "…", "label": "Bus 64 places", "plate": "DK-4821-BF", "capacity": 64, "seatsPerRow": 4,
    "blockedSeats": [] }
```

Specific errors

Status	code	When
409	PLATE_TAKEN	Already registered.

Create a departure (draft)

POST `/api/v1/org/{orgId}/departures`

Nothing is sold until it's published. Stops in route order; the first pickup is the departure.

Access Bearer token + role **owner** or **manager** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).

Request body (JSON)

Field	Type	Required	Rules / meaning
`vehicleId`	ObjectId	yes	
`direction`	aller | retour	yes	
`from, to`	string	yes	Cities on the card.
`pickups, dropoffs`	[{ place, at }]	yes	1–15 each, times in order.
`price`	integer FCFA	yes	
`minPassengers`	integer	yes	0 = confirmed at once.
`confirmBy, bookingClosesAt`	ISO dates	no	Defaults: 24 h and 30 min before.

Example request

```
{ "vehicleId": "…", "direction": "aller", "from": "UGB", "to": "Dakar", "price": 3500,
    "minPassengers": 40,
  "pickups": [ { "place": "UGB · Village E", "at": "2026-11-02T14:50:00Z" } ],
  "dropoffs": [ { "place": "Dakar · Colobane", "at": "2026-11-02T20:25:00Z" } ] }
```

Success response · 201 Created

```
the departure, "status": "draft", stops numbered p1…, d1…
```

Edit a draft

PUT `/api/v1/org/{orgId}/departures/{id}`

Drafts only: once published, price, stops and seats are what people paid for.

Access Bearer token + role **owner** or **manager** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).
`id`	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`(same as create)`		yes	

Example request

```
same body as create
```

Success response · 200 OK

```
the departure
```

Specific errors

Status	code	When
`409`	`DEPARTURE_PUBLISHED`	Cancel it and create another.

Publish (open bookings)

POST `/api/v1/org/{orgId}/departures/{id}/publish`

Students see it and can book.

Access Bearer token + role **owner** or **manager** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).
`id`	ObjectId (24 hex)	yes	

Example request

```
POST /api/v1/org/{orgId}/departures/{id}/publish
```

Success response · 200 OK

```
the departure, "status": "published"
```

Cancel a departure

POST `/api/v1/org/{orgId}/departures/{id}/cancel`

Before the bus leaves. Every ticket is cancelled and refunded 100 %, holders are notified, unpaid holds released.

Access Bearer token + role **owner** or **manager** in the organization

Path parameters

Name	Type	Required	Description
`orgId`	ObjectId	yes	The organization (the caller must be a member, else 404).
`id`	ObjectId (24 hex)	yes	

Request body (JSON)

Field	Type	Required	Rules / meaning
`reason`	string 3–200	yes	

Example request

```
{ "reason": "Panne du bus" }
```

Success response · 200 OK

```
{ "refunded": 12 }
```

Admin back office

There is no web back office yet: these routes are used from Swagger UI at `http://localhost:3000/docs` ("Authorize" with an admin access token). They onboard organizers (accounts, organizations, owners). Every change is written to the audit log.

Method	Path	Goal
`POST`	`/api/v1/admin/users`	Create an account for an organizer or staff member
`PUT`	`/api/v1/admin/users/{id}/platform-roles`	Set a user's platform roles
`GET`	`/api/v1/admin/organizations`	List organizations
`POST`	`/api/v1/admin/organizations`	Create an organization
`PATCH`	`/api/v1/admin/organizations/{id}`	Update an organization
`PUT`	`/api/v1/admin/organizations/{id}/members`	Give someone a role in an organization
`DELETE`	`/api/v1/admin/organizations/{id}/members/{userId}`	Remove someone from an organization

Create an account for an organizer or staff member

POST **`/api/v1/admin/users`**

Creates a non-student account (no student profile), for example the boss of a transport company. The account has **no password**: the person sets one with "mot de passe oublié" (code sent to their phone), so the admin never knows it.

Access Bearer token + platform role **admin**

Request body (JSON)

Field	Type	Required	Rules / meaning
`phone`	string	yes	Senegalese mobile, 9 digits starting with 70, 71, 75, 76, 77 or 78. Spaces and the +221 / 00221 prefix are accepted: "77 123 45 67", "+221771234567".
`firstName`	string	yes	1–60 letters.
`lastName`	string	yes	1–60 letters.

Example request

```
{ "phone": "77 000 00 01", "firstName": "Modou", "lastName": "Khoulé" }
```

Success response · 201 Created

```
{ "id": "6ae5dc1e15fb772d267ad562" }
```

Specific errors

Status	code	When
`403`	`FORBIDDEN`	Caller is not admin.
`409`	`PHONE_TAKEN`	An account already uses this phone (just add it as a member instead).

Set a user's platform roles

PUT `/api/v1/admin/users/{id}/platform-roles`

Grants or removes back-office roles. The list **replaces** the current one ([] removes all). Takes effect on the user's next request.

Access Bearer token + platform role **admin**

Path parameters

Name	Type	Required	Description
`id`	string (24 hex)	yes	User id.

Request body (JSON)

Field	Type	Required	Rules / meaning
`roles`	array	yes	Any of: admin, content_editor, resto_u_staff, lost_office, academic_editor, finance.

Example request

```
{ "roles": ["content_editor"] }
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
403	FORBIDDEN	Caller is not admin.
404	NOT_FOUND	Unknown user.

List organizations

GET `/api/v1/admin/organizations`

All organizations (caravan operators, associations, restaurants) with their level and status, sorted by name.

Access Bearer token + platform role **admin**

Example request

```
(no body)
```

Success response · 200 OK

```
[ {
    "id": "6ae5dc1e15fb772d267ad55d",
    "slug": "amicale-zig",
    "name": "Amicale de Ziguinchor",
    "category": "Amicale",
    "capabilities": ["caravans"],
    "level": "verifie",
    "rating": 4.7,
    "trips": 5,
    "status": "active",
    "hasPayoutAccount": true,
    "email": "amicale.ziguinchor@ugb.edu.sn",
    "logoUrl": null
  }, ... ]
```

Specific errors

Status	code	When
403	FORBIDDEN	Caller is not admin.

Create an organization

POST `/api/v1/admin/organizations`

Registers a company, an amicale or a restaurant after its documents were checked. Then add its owner with PUT /admin/organizations/{id}/members.

Access Bearer token + platform role **admin**

Request body (JSON)

Field	Type	Required	Rules / meaning
slug	string	yes	Short unique id: lowercase letters, digits, hyphens (2–60), e.g. "khoule".
name	string	yes	2–80 characters.
category	"Entreprise" | "Amicale" | "Transporteur" | "Restaurant"	yes	What the organization is (replaces the former "kind").
capabilities	array of "caravans" | "restaurant"	yes	What it may publish.
level	"nouveau" | "verifie" | "confirme"	no	Default "nouveau". Shown to students on each departure.
payoutPhone	string	no	Wave / Orange Money number that receives organizer claims.
email	string	no	Contact address. Can also be added later.

Example request

```
{
  "slug": "ndar",
  "name": "GIE Ndar Transport",
  "category": "Transporteur",
  "capabilities": ["caravans"],
  "level": "nouveau",
  "payoutPhone": "77 555 01 01",
  "email": "contact@ndar-transport.sn"
}
```

Success response · 201 Created

```
{
  "id": "6ae5dc1e15fb772d267ad55d",
  "slug": "amicale-zig",
  "name": "Amicale de Ziguinchor",
  "category": "Amicale",
  "capabilities": ["caravans"],
  "level": "verifie",
  "rating": 4.7,
  "trips": 5,
  "status": "active",
  "hasPayoutAccount": true,
  "email": "amicale.ziguinchor@ugb.edu.sn",
  "logoUrl": null
}
```

Specific errors

Status	code	When
400	VALIDATION_ERROR	Bad slug, category, phone or e-mail.
409	SLUG_TAKEN	Slug already used.

Update an organization

PATCH `/api/v1/admin/organizations/{id}`

Changes the name, the verification level (until ratings exist), the status (suspending closes the organizer space for all its members) or the payout number. Only the fields sent are changed.

Access Bearer token + platform role **admin**

Path parameters

Name	Type	Required	Description
id	string (24 hex)	yes	Organization id.

Request body (JSON)

Field	Type	Required	Rules / meaning
name	string	no	
level	"nouveau" | "verifie" | "confirme"	no	
status	"pending" | "active" | "suspended"	no	
payoutPhone	string | null	no	null removes it.
email	string | null	no	Contact address; null removes it.

Example request

```
{ "level": "verifie" }
```

Success response · 200 OK

```
The updated organization (same shape as above).
```

Specific errors

Status	code	When
404	NOT_FOUND	Unknown organization.

Give someone a role in an organization

PUT `/api/v1/admin/organizations/{id}/members`

Same as the owner's team route, but the admin may also appoint **owners**. This is how an organizer gets access to the organizer space.

Access Bearer token + platform role **admin**

Path parameters

Name	Type	Required	Description
id	string (24 hex)	yes	Organization id.

Request body (JSON)

Field	Type	Required	Rules / meaning
`phone`	string	yes	Senegalese mobile, 9 digits starting with 70, 71, 75, 76, 77 or 78. Spaces and the +221 / 00221 prefix are accepted: "77 123 45 67", "+221771234567".
`roles`	array of owner | manager | scanner | staff	yes	1 to 4 roles.

Example request

```
{ "phone": "77 000 00 01", "roles": ["owner"] }
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
404	NOT_FOUND	Unknown organization.
422	USER_NOT_FOUND	No account with this phone (create it with POST /admin/users).

Remove someone from an organization

DELETE `/api/v1/admin/organizations/{id}/members/{userId}`

Revokes any member, owners included.

Access Bearer token + platform role **admin**

Path parameters

Name	Type	Required	Description
`id`	string (24 hex)	yes	Organization id.
`userId`	string (24 hex)	yes	

Example request

```
(no body)
```

Success response · 204 No Content

```
(empty)
```

Specific errors

Status	code	When
404	NOT_FOUND	Not an active member.

Health

For the load balancer and monitoring, not for the app. These two routes are **not** under /api/v1.

Method	Path	Goal
GET	/health/live	Is the process alive?
GET	/health/ready	Can this instance serve traffic?

Is the process alive?

GET `/health/live`

Answers as long as the process runs. Used by Docker's HEALTHCHECK to restart a frozen container.

Access Public (no token)

Example request

```
(no body)
```

Success response · 200 OK

```
{ "status": "ok" }
```

Can this instance serve traffic?

GET `/health/ready`

Checks MongoDB and Redis. Answers 503 during shutdown so the load balancer stops sending requests before the instance stops.

Access Public (no token)

Example request

```
(no body)
```

Success response · 200 OK

```
{ "status": "ok", "checks": { "mongo": "ok", "redis": "ok" } }

// 503 when a dependency fails or the instance is shutting down:
{ "status": "unavailable", "checks": { "mongo": "ok", "redis": "fail" } }
```

Appendix: planned endpoints

Not available yet. These endpoints are designed (docs/04-api.md and docs/09-decisions.md) and will be built module by module. Paths are relative to /api/v1 and may still change; this document will be regenerated as each module ships.

Home & content

Method	Path	Goal
GET	/home	One call for the Accueil tab: alert, next caravan, Sunu Taxi, Resto U, ads, À la une
GET	/announcements	Full list of headlines
POST/PATCH	/admin/announcements…	Publish, schedule, archive announcements

Food

Method	Path	Goal
GET	/resto-u/menus	Resto U menus with stock status
GET	/restaurants, /restaurants/{id}	Private restaurants by campus, menus, drinks, open/closed
POST	/orders/quote	Server-side price of a cart
POST	/orders	Place an order and start payment
GET	/orders, /orders/{id}	My orders and live status
GET/POST	/org/{orgId}/orders…	Restaurant queue, status changes

Lost & found

Method	Path	Goal
GET/POST	/lost-found/items	Search and declare items
POST	/lost-found/card-scans	Found student card: warn its owner
POST	/lost-found/items/{id}/claims	"C'est à moi" with a proof
POST	/lost-found/claims/{id}/accept|reject	Finder decides
—	conversation type lost_claim	Finder ↔ owner chat, on the existing Chat endpoints
POST	/media	Photo upload (EXIF removed)
PUT	/me/avatar, /org/{orgId}/logo	Set the profile picture / organization logo from an uploaded photo

Campus, courses, scholarships

Method	Path	Goal
GET	/places, /places/{id}	Campus places, map, directions
GET	/me/timetable	Weekly timetable with moved classes
GET	/scholarships/next-payment	Next scholarship payment